



4. variable length positional argument (*args)

Sometimes it is not fixed that how many positional arguments to pass, pass, at that time we use variable length positional argument

- It takes *args as a tuple

Eg.

Positional Argument

```
def greet(name, age):
```

```
    print("my name is:", name "and age is:", age)
```

```
greet("bhakti", 21)
```

```
def calculate_sum(*args):
```

```
    print(args)
```

```
    print(type(args))
```

```
    return sum(args)
```

```
result = calculate_sum(1, 2, 3)
```

```
print(result)
```

→ variable length positional argument

```
result = calculate_sum(1, 2, 3, 4, 5, 6, 7)
```

5. Variable length keyword Argument- (**kwargs)

- using variable length keyword argument, we can pass many keyword arguments to the function
- **kwargs can take as many keyword arguments.
- The keywords arguments we pass, we take as a key-value pair.

3 - Inside function, kwargs act as a dictionary.



Eg.

```
def print_details(**kwargs):
    print(kwargs)
    print(type(kwargs))
    for k, v in kwargs.items():
        print(k, v)

print_details(name="rahul", age=25, city="pune")
> {'name': 'rahul', 'age': 25, 'city': 'pune'}
<class 'dict'>
name rahul
age 25
city pune
```

◆ Functional Programming:

1] lambda 2] Map 3] Filter 4] Reduce.

1] Lambda Function -

- Lambda is a anonymous function - nameless function.
- It is defined using lambda keyword.

Eg: Normal Function -

```
def fun-name():
```

```
    _____
    _____
    fun-name()
```

lambda function - Syntax :-

lambda arguments : expression



Eg.

```
lambda x, y : x + y
```

- Lambda function is a simple through throw away function that we are not going to use in future

~ characteristics of lambda function:

1. Anonymous function - nameless
2. single expression - one line
3. return value - automatically returns value

Q1. create a single lambda function that adds two numbers.

Normal Function :

```
def add(x, y):
    sum = x + y
    return result
ans = add(5, 10)
print(ans)
> 15
```

Lambda Function :

```
add = lambda x, y : x + y
print(add(5, 10))
```

Q2 check whether the number is even or not

normal Function :-

```
def is-even(x):
    if (x % 2 == 0):
        print("True")
```

```
is-even(6)
```

```
> True
```

Lambda Function :-

```
is-even = lambda x : x % 2 == 0
```

```
print(is-even(6))
```

```
> True
```



Q3. Square of no.

```
Square = lambda x : x**2
print(square(5))
> 25
```

Q4. Concatenation of string using lambda.

```
Concat = lambda str1, str2 : str1 + str2
print(concat("hello", "World!"))
> hello world!
```

2] Map Function -

- Map Function in python is useful that applies a given function to each item of iterable (list, tuple, set, range) and creates a new iterable with result.

Syntax :

map(function, iterable)

Function - normal func, lambda func

iterable - list, tuple, set, range

eg.

```
function → def square(x):
            return x**2
```

```
iterable (list) → [1, 2, 3, 4, 5]
```

```
> 1 4 9 16 25 ← result (map object)
```



```
list(map_obj)
```

```
> [1, 4, 9, 16, 25]
```

- When we apply map function it return we get map object that we convert into iterable such as list, set, tuple, etc.

Q1. Square of no-s using Map function

```
numbers = [1, 2, 3, 4, 5]
```

```
def square(x):
```

```
    return x**2
```

```
Sq-nos = map(numbers, square)
```

```
print(Sq-nos)
```

```
> <map object at 0x00001DFB9CC9F00>
```

```
numbers = [1, 2, 3, 4, 5]
```

```
def square(x):
```

```
    return x**2
```

```
Sq-nos = map(square, numbers)
```

```
print(list(Sq-nos))
```

```
> [1, 4, 9, 16, 25]
```

Q2. ~~add to every~~ add 10 in every value & result new list using map

```
numbers = [1, 2, 3, 4, 5]
```

```
def add(x):
```

```
    return x + 10
```

```
result = list(map(add, numbers))
```

```
print(result)
```

```
> [11, 12, 13, 14, 15]
```